EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Week 5a: Deep-Dive Hardware Architecture: RP2350 PADS_BANK0, IO_BANK0, SIO, and the GPIO Coprocessor (GPIOC)

ARM Cortex-M33 • GDB • Ghidra

October 19, 2026

For educational and defensive security research only

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

What You'll Learn This Week

By the end of this tutorial, you will be able to:

- Master the physical silicon reality of General Purpose Input/Output (GPIO) on the RP2350 microcontroller from Chapter 9 of the datasheet.
- Understand why microcontroller pins are never simple wires, dissecting the split between the internal 1.1V digital core domain (Vcore / DVDD) and the external 3.3V analog pad ring domain (Vio / IOVDD).
- Decouple the two independent hardware layers: electrical pad configuration (`PADS_BANK0` at `0x40038000`) versus digital crossbar multiplexing (`IO_BANK0` at `0x40028000`).
- Dissect every bitfield of `PADS_BANK0` : Output Disable (`OD`), Input Enable (`IE`), Pad Isolation (`ISO`), Drive Strength (2/4/8/12 mA), Pull-Up (`PUE`), Pull-Down (`PDE`), Schmitt Trigger (`SCHMITT`), and Slew Rate (`SLEWFAST`).
- Discover why unconfigured RP2350 pins are physically isolated by default (`ISO = 1`) and why digital input buffers are disconnected (`IE = 0`) to prevent catastrophic CMOS shoot-through leakage current.
- Master RP2350 **Bus Keeper Mode**, activated by setting both `PUE = 1` and `PDE = 1` to weakly latch floating line states without power dissipation.
- Navigate the digital crossbar switchboard in `IO_BANK0` : function selection (`FUNCSEL 0..31`), peripheral routing (UART, SPI, I2C, PWM, SIO, PIO), and signal overrides (`OUTOVER` , `OEOVER` , `INOVER` , `IRQOVER`).
- Analyze edge-sensitive and level-sensitive hardware interrupt generation across `proc0` , `proc1` , and `dormant_wake` .
- Understand the hardware Single-Cycle I/O (`SIO` at `0xd0000000`) block and contrast standard MMIO load/store operations against the RP2350 **GPIO Coprocessor (GPIOC)** on ARM Cortex-M33 Coprocessor Port `p0` (Section 3.6.1 and Section 9.9).
- Trace Thumb-2 coprocessor instructions (`mcrr p0` , `mcr p0` , `mrrc p0`) that set, clear, toggle, and configure GPIO output-enables in a single clock cycle with zero memory bus arbitration.
- Deconstruct the assembly implementation in `0x000b_integer-data-type.c` , performing a cycle-accurate trace of `asm_init_gpio_range()` and `asm_blink_pin()` .
- Eliminate the disassembler "hardware MMIO blind spot" using PyCortexMDebug in GDB for live dynamic peripheral struct inspection.

- Import CMSIS-SVD hardware definitions into Ghidra using SVD-Loader-Ghidra to reconstruct clear, readable decompilation from raw binary firmware.

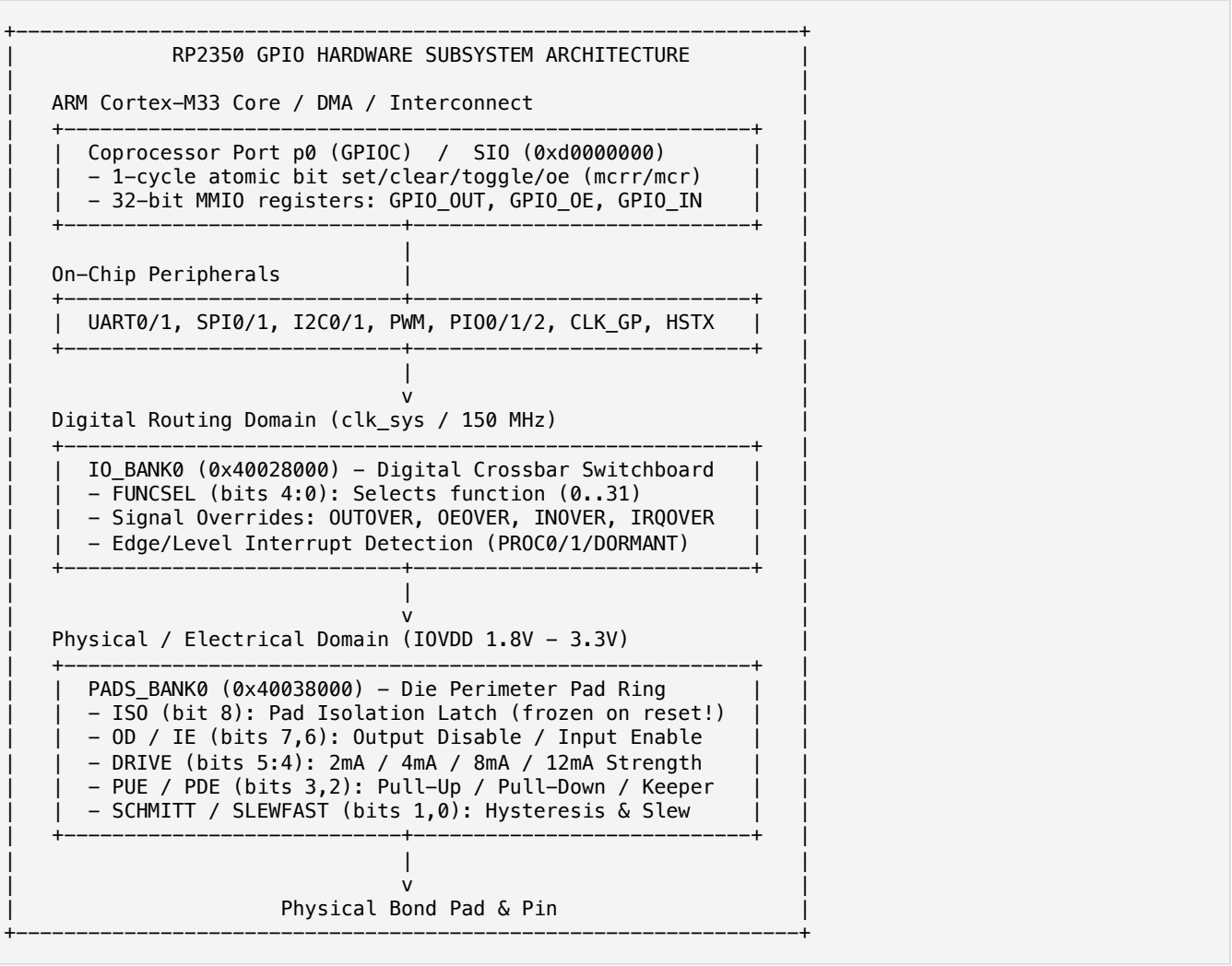
Part 1: Why Microcontroller Pins Are Not Simple Wires

The Silicon Reality: Core Logic vs. The Pad Ring

In software, a developer writes `gpio_put(16, 1)` and visualizes a direct wire connecting a CPU register bit to an external copper pin. In silicon engineering, this concept is completely false.

A modern microcontroller die like the RP2350 is divided into two radically distinct physical and electrical domains:

1. **The Internal Digital Core Domain (1.1V nominal DVDD):** The central silicon die contains the dual ARM Cortex-M33 processors, the bus fabric, static RAM, cache controllers, and digital peripheral state machines (UART, SPI, PIO). These circuits are fabricated using ultra-thin, low-voltage nanoscale transistors designed exclusively for high switching speeds (150 MHz) and ultra-low power consumption. These microscopic gates cannot tolerate voltages above 1.21V. Connecting an external 3.3V signal directly to a core gate would instantly cause dielectric oxide breakdown, permanently melting the transistor.
2. **The External Pad Ring Domain (1.8V - 3.3V nominal IOVDD):** Surrounding the perimeter of the die is the **Pad Ring**. The pad ring consists of specialized, high-voltage analog and electrical interface cells ("pad cells") fabricated with thick-oxide transistors. These cells translate internal 1.1V digital logic signals up to external 3.3V board voltages, provide electrostatic discharge (ESD) protection diodes, supply high output drive currents (up to 12 mA) to charge external capacitive PCB traces, and filter noisy incoming analog edges.



The Physical Hazards of Uncontrolled I/O

Why can't a microcontroller provide a single "GPIO register" where bit n directly drives pin n? Because every physical pin faces severe physical and electrical constraints:

- **CMOS Shoot-Through Current:** When a CMOS digital input buffer encounters a voltage near mid-rail (e.g., 1.65V on a 3.3V rail), both the upper P-channel and lower N-channel MOSFETs turn partially ON simultaneously. This creates a low-resistance path directly from `IOVDD` to ground, dissipating substantial leakage current and generating heat. To prevent this on unused or analog pins (such as ADC inputs), the chip must provide a hardware switch to completely disconnect the input buffer (`IE = 0`).
- **Electrostatic Discharge (ESD):** External human touch or inductive spikes can inject thousands of volts into a pin. Pad cells integrate dual clamping diodes to shunt overvoltage spikes safely into power and ground rails.
- **Power Domain Collapse during Deep Sleep:** When the RP2350 enters low-power dormant sleep, the 1.1V core voltage can be completely powered down by the Power Manager (`POWMAN`). If the pad ring were directly connected to the core, the unpowered core outputs would float, causing external actuators, transceivers, or power supplies to turn on uncontrollably. The pad ring must therefore integrate **Pad Isolation Latches (`ISO`)** that freeze the external electrical state even when the CPU and digital core no longer exist!

To solve these orthogonal challenges, the RP2350 divides GPIO into a clean three-tier hardware hierarchy:

- `PADS_BANK0` (`0x40038000`): Governs the electrical pad cells along the silicon die perimeter.
- `IO_BANK0` (`0x40028000`): Governs digital routing, multiplexing, and signal overrides.
- `SIO` (`0xd0000000`) & **Coprocessor Port** `p0`: Governs single-cycle processor execution.

Part 2: PADS_BANK0 Deep Dive: The Electrical Pad Ring

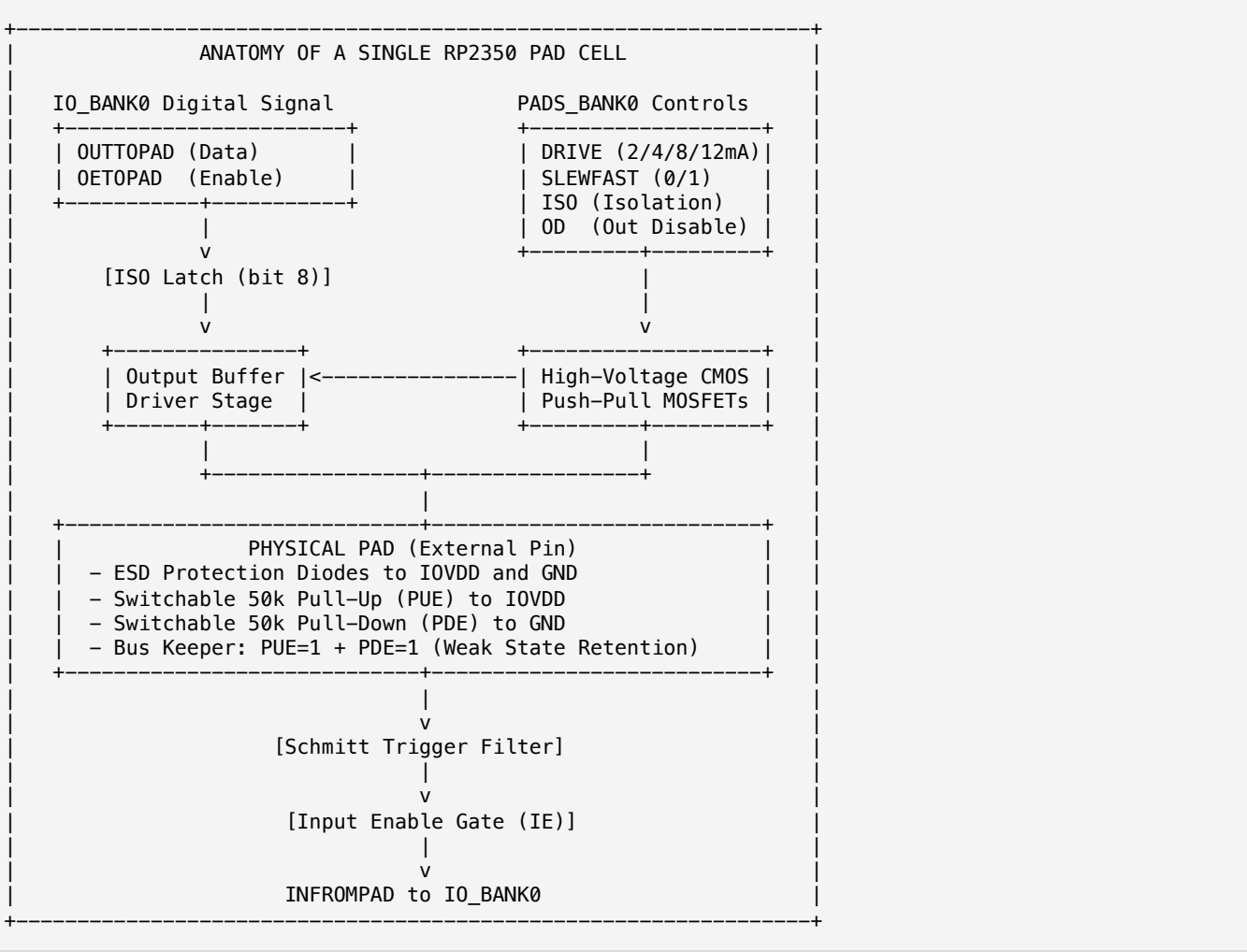
Physical Placement and Register Map

The User Bank Pad Control registers reside at base address `0x40038000` (`PADS_BANK0_BASE`).

According to **Chapter 9.11.3 (Table 851)** of the RP2350 Datasheet:

- Offset `0x00`: `VOLTAGE_SELECT` (Bank-wide voltage threshold: `0 = 3.3V`, `1 = 1.8V`).
- Offset `0x04`: `GPIO0` pad control register.
- Offset `0x08`: `GPIO1` pad control register.
- ...
- Offset `0x04 + (4 * x)`: `GPIOx` pad control register.

Each 32-bit register controls the analog and physical circuitry of exactly one pin.



The PADS_BANK0 Bitfield Breakdown

Every 32-bit `GPIOx` register in `PADS_BANK0` has the exact bitfield structure defined in **Table 853**:

```
+-----------------------------------------------------------------+
|       PADS_BANK0: GPIOx REGISTER BITFIELDS (Offset: 0x04 + 4*x) |
|                                                                 |
|   31            9   8   7   6   5   4   3   2   1   0          |
|  +--------------+---+---+---+-------+---+---+---+---+          |
|  |   Reserved    |ISO|OD |IE | DRIVE |PUE|PDE|SCH|SLW|          |
|  +--------------+---+---+---+-------+---+---+---+---+          |
|                                                                 |
|  Bit 8:   ISO       - Pad isolation latch (1 = isolated/frozen) |
|  Bit 7:   OD        - Output disable (1 = output driver off)    |
|  Bit 6:   IE        - Input enable (1 = digital input buffer on)|
|  Bits 5:4 DRIVE     - Drive strength (00=2mA, 01=4mA, 10=8mA,   |
|                                  11=12mA)                       |
|  Bit 3:   PUE       - Pull-up resistor enable (~50 kOhm)        |
|  Bit 2:   PDE       - Pull-down resistor enable (~50 kOhm)      |
|  PUE & PDE == 1     - Bus Keeper Mode (weakly holds last state) |
|  Bit 1:   SCHMITT   - Schmitt trigger hysteresis filter         |
|  Bit 0:   SLEWFAST  - Slew rate control (0 = Slow, 1 = Fast)    |
+-----------------------------------------------------------------+
```

Let us examine each bitfield with silicon-level precision:

1. ISO (Bit 8) - Pad Isolation Control

- **Reset Value:** `0x1` (Isolated!).
- **Function:** When `ISO = 1`, an analog transmission gate isolates the pad from the switched core domain. The output driver state, output enable, and pull resistors remain frozen at their last latched values.
- **The Startup Gotcha:** Upon microcontroller power-up or reset, `ISO` resets to `1` across all pads! Before any pin can be driven or read by software, **the application software must explicitly clear the `ISO` bit**. If you attempt to configure `IO_BANK0` or `SIO` while `ISO = 1`, the external pin will remain totally unresponsive!

2. OD (Bit 7) - Output Disable

- **Reset Value:** `0x0`.
- **Function:** Master hardware output disable. When `OD = 1`, the pad's output driver transistors are completely turned OFF, forcing the pin into high-impedance (High-Z) mode.
- **Priority:** `OD` has absolute silicon priority over any output-enable signal emitted by internal peripherals (UART, SPI, PWM, SIO). Even if the UART hardware attempts to transmit, setting `OD = 1` silences the pin.

3. IE (Bit 6) - Input Enable

- **Reset Value:** `0x0` (Disabled!).
- **Function:** Master hardware gate for the digital input buffer.
- **The Analog Protection Rule:** When a pin is used as an analog input (e.g., ADC pins GPIO 26..29), or when a pin is left disconnected/floating, `IE` must remain `0`. Disabling the input buffer prevents floating voltages from causing CMOS shoot-through currents across internal transistor pairs.
- **The Input Gotcha:** If you want to read digital input from a button or serial line using SIO (`sio_hw->gpio_in`), you **must explicitly set `IE = 1`** in `PADS_BANK0`. If `IE = 0`, reading `GPIO_IN` will always return `0`, regardless of the voltage applied to the physical pin!

4. DRIVE (Bits 5:4) - Output Drive Strength

- **Reset Value:** `0x1` (4 mA).
- **Options:** `00 = 2mA`, `01 = 4mA`, `10 = 8mA`, `11 = 12mA`.
- **Silicon Implementation:** Inside the pad cell, the output push-pull stage consists of parallel arrays of PMOS (pull-up) and NMOS (pull-down) transistors. Selecting 2 mA enables a single transistor pair; selecting 12 mA switches on all parallel pairs, dramatically lowering the driver's output resistance (Ron).
- **Engineering Trade-off:** High drive strength (12 mA) charges external trace capacitance rapidly, enabling high-speed 50 MHz+ SPI clocks. However, it generates sharp current transients (dI/dt), leading to ground bounce, ringing, and electromagnetic interference (EMI). Low drive strength (2 mA) softens transitions, producing clean, low-noise waveforms on sensitive sensor buses.

5. PUE (Bit 3) & PDE (Bit 2) - Pull-Up and Pull-Down Resistors

- **Reset Value:** `PUE = 0`, `PDE = 1` (Pads reset with weak pull-downs active!).
- **Electrical Specification:** Nominal internal resistance of approximately 50 kOhm to `IOVDD` (PUE) or ground (PDE).
- **Bus Keeper Mode (Section 9.6.1):** What happens if software sets both `PUE = 1` and `PDE = 1` simultaneously? On older microcontrollers, this would create an illegal cross-conduction voltage divider dissipating power. On the RP2350, setting both bits activates **Bus Keeper Mode**! In this mode:
  - If the pad's voltage is High, the pad pulls weakly Up.
  - If the pad's voltage is Low, the pad pulls weakly Down. When an external device stops driving a shared bus (e.g., a bidirectional data line in tri-state), the Bus Keeper holds the last valid logic level indefinitely, preventing the line from floating into mid-rail states without drawing active power!

6. SCHMITT (Bit 1) - Schmitt Trigger Enable

- **Reset Value:** `0x1` (Enabled).
- **Function:** Adds voltage hysteresis to the input comparator (VIH ~ 2.0V, VIL ~ 0.8V). If an incoming digital signal has a slow rise time or is corrupted by high-frequency noise, the Schmitt trigger prevents false multi-triggering and oscillation at the digital threshold.

7. SLEWFAST (Bit 0) - Slew Rate Control

- **Reset Value:** `0x0` (Slow).
- **Function:** `0 = Slow`, `1 = Fast`. Slow slew rate actively limits the edge rate (dV/dt) of the output buffer, minimizing radiated emissions and transmission-line reflections on unterminated PCB traces. Fast slew rate is required for high-frequency interfaces like QSPI, HSTX, or high-speed UART.

Part 3: IO_BANK0 Deep Dive: The Digital Crossbar Switchboard

The Multiplexer Matrix (`0x40028000`)

Once a signal passes through the electrical pad ring, it enters the digital core domain. Here sits `IO_BANK0`, located at base address `0x40028000` (`IO_BANK0_BASE`).

`IO_BANK0` functions as an enormous digital crossbar switchboard. The RP2350 contains dozens of peripheral controllers:

- Two SPI controllers (`SPI0`, `SPI1`)
- Two UART controllers (`UART0`, `UART1`)
- Two I2C controllers (`I2C0`, `I2C1`)

- Up to 12 dual-channel PWM slices (`PWM0` through `PWM11`)
- Three Programmable I/O blocks (`PIO0` , `PIO1` , `PIO2`)
- Single-Cycle I/O (`SIO`)
- General Purpose Clock outputs (`CLK_GP`)
- High-Speed Serial Transmitter (`HSTX`)

None of these peripherals own any dedicated physical pins! Instead, every peripheral signal is routed through `IO_BANK0` , which selects which peripheral connects to which pad.

Register Structure per Pin

In `IO_BANK0` , every GPIO pin occupies an **8-byte stride** containing two 32-bit registers (Section 9.11.1, Table 649):

```
GPIOx_STATUS Offset = 0x000 + (8 * x)
GPIOx_CTRL Offset   = 0x004 + (8 * x)
```

For example, for GPIO 16:

```
GPIO16_CTRL Offset = 0x004 + (8 * 16) = 0x004 + 0x080 = 0x084
Physical Address   = 0x40028000 + 0x084 = 0x40028084
```

The GPIOx_CTRL Register: Function Selection & Overrides

```
+-------------------------------------------------------------------+
|       IO_BANK0: GPIOx_CTRL REGISTER BITFIELDS (Offset: 0x04+8*x)|
|                                                                   |
|   31 30 29 28 27     18 17 16 15 14 13 12 11      5 4        0  |
|  +-----+-----+---------+-----+-----+-----+---------+---------+  |
|  |Rsvd |IRQO |Reserved |INOV |OEOV |OUTOV|Reserved | FUNCSEL |  |
|  +-----+-----+---------+-----+-----+-----+---------+---------+  |
|                                                                   |
|  Bits 4:0   FUNCSEL - Function Select (0..31, 5=SIO, 31=NULL)   |
|  Bits 13:12 OUTOVER - Output signal override (0=norm, 1=invert, |
|                       2=force low, 3=force high)                 |
|  Bits 15:14 OEOVER  - Output enable override (0=norm, 1=invert, |
|                       2=force disable, 3=force enable)           |
|  Bits 17:16 INOVER  - Input signal override (0=norm, 1=invert,  |
|                       2=force low, 3=force high)                 |
|  Bits 29:28 IRQOVER - Interrupt override (0=norm, 1=invert,     |
|                       2=force low, 3=force high)                 |
+-------------------------------------------------------------------+
```

According to **Table 651**, the bitfields of `GPIOx_CTRL` provide ultimate routing and logic control:

1. FUNCSEL (Bits 4:0) - Function Select

A 5-bit multiplexer index determining which peripheral controls the pin:

- `0x00` : JTAG
- `0x01` : SPI
- `0x02` : UART
- `0x03` : I2C
- `0x04` : PWM
- `0x05` **: SIO (Single-Cycle I/O - CPU / Coprocessor Software Control)**

- `0x06` : PIO0
- `0x07` : PIO1
- `0x08` : PIO2
- `0x09` : Clock / Auxiliary
- `0x1f` (`31`): **NULL / Disconnected** (Reset Value!)

Upon reset, `FUNCSEL` is set to `0x1f` (NULL). The pin is completely disconnected from all internal peripherals. To control a pin via CPU software or inline assembly, you must write `5` to `FUNCSEL`.

2. Signal Overrides (OUTOVER, OEOVER, INOVER, IRQOVER)

The RP2350 allows hardware hackers and defensive engineers to intercept, invert, or force signals between peripherals and pads without reconfiguring peripheral registers:

Override Field	Bits	Values & Behavior
`OUTOVER`	`13:12`	`0 = Normal` (Peripheral drives data) `1 = Invert` (Invert peripheral data) `2 = Force Low` (Drive 0 to pad) `3 = Force High` (Drive 1 to pad)
`OEOVER`	`15:14`	`0 = Normal` (Peripheral drives OE) `1 = Invert` (Invert OE) `2 = Force Disable` (Disable output driver) `3 = Force Enable` (Enable output driver)
`INOVER`	`17:16`	`0 = Normal` (Pad drives peripheral) `1 = Invert` (Invert pad data to peripheral) `2 = Force Low` (Peripheral sees 0) `3 = Force High` (Peripheral sees 1)
`IRQOVER`	`29:28`	`0 = Normal` (Pad drives interrupt) `1 = Invert` (Invert interrupt logic) `2 = Force Low` (Deassert IRQ) `3 = Force High` (Assert IRQ)

Security/Hacking Implication: If a target firmware runs a proprietary UART protocol on GPIO 0/1, an analyst can invert the transmitted serial stream in hardware by writing `0x1` to `OUTOVER`, without the target firmware ever detecting that its serial framing has been altered!

The GPIOx_STATUS Register: Real-Time Silicon Probing

The read-only `GPIOx_STATUS` register (+ `0x000`) provides a live digital probe inside the chip (Table 650):

- `INFROMPAD` **(Bit 17):** The raw digital logic level coming from the pad cell (after Schmitt trigger, before any `INOVER` inversion).
- `OETOPAD` **(Bit 13):** The effective Output Enable signal being transmitted to the pad cell after all overrides.
- `OUTTOPAD` **(Bit 9):** The effective Output Data signal being transmitted to the pad cell after all overrides.
- `IRQTOPROC` **(Bit 26):** The interrupt status signal sent to the CPU NVIC.

Part 4: SIO vs. The RP2350 GPIO Coprocessor (GPIOC)

The Single-Cycle I/O (SIO) Architecture

When a pin's `FUNCSEL` is set to `5` in `IO_BANK0`, it connects to the **SIO (Single-Cycle I/O)** block, located at base address `0xd0000000` (`SIO_BASE`).

SIO provides fast, non-blocking registers that allow the ARM CPU to manipulate GPIOs in 1 clock cycle:

- `GPIO_OUT` (`0xd0000010`): Direct 32-bit output value.
- `GPIO_OUT_SET` (`0xd0000014`): Atomic bit set (writing a `1` sets the corresponding pin High).
- `GPIO_OUT_CLR` (`0xd0000018`): Atomic bit clear (writing a `1` clears the corresponding pin Low).
- `GPIO_OUT_XOR` (`0xd000001c`): Atomic bit toggle (writing a `1` inverts the corresponding pin).
- `GPIO_OE` (`0xd0000020`), `GPIO_OE_SET`, `GPIO_OE_CLR`, `GPIO_OE_XOR`: Output Enable registers.
- `GPIO_IN` (`0xd0000004`): Direct 32-bit digital input read.

The Memory Bus Overhead Bottleneck

While SIO registers respond in a single clock cycle, interacting with them via standard ARM memory-mapped instructions (`ldr`, `str`) suffers from severe software overhead:

1. **Address Materialization:** Cortex-M33 instructions cannot embed a 32-bit address like `0xd0000014` directly in a single 16-bit Thumb opcode. The compiler must generate an `ldr r0, =0xd0000014` instruction, pulling the 32-bit address from a Flash literal pool (taking 2 cycles plus cache/wait-states).
2. **Register Pressure:** The CPU must allocate a general-purpose register (`r0`) to hold the address and a second register (`r1`) to hold the bitmask (`1 << pin`).
3. **Bus Arbitration:** Even though SIO is single-cycle, the `str` instruction still arbitrates across the internal AHB-Lite bus matrix.

```
+-------------------------------------------------------------------+
|          SIO MMIO ACCESS VS. GPIOC COPROCESSOR PORT P0            |
|                                                                   |
|   Standard SIO Access (3 to 5 Instructions / Bus Bottleneck)      |
|   +-----------------------------------------------------------+   |
|   |  ldr  r0, =0xd0000000      ; Load 32-bit SIO base         |   |
|   |  movs r1, #1               ; Load bitmask                 |   |
|   |  lsls r1, r1, #16          ; Shift to pin 16              |   |
|   |  str  r1, [r0, #0x14]      ; Write to GPIO_OUT_SET        |   |
|   |  => Requires register allocation, memory bus arbitration  |   |
|   +-----------------------------------------------------------+   |
|                                                                   |
|   RP2350 GPIOC Coprocessor Access (1 Instruction / Zero Bus)      |
|   +-----------------------------------------------------------+   |
|   |  mcrr p0, #4, r4, r5, c0   ; gpioc_bit_out_put(pin,val)   |   |
|   |  - r4 = pin number (16)                                   |   |
|   |  - r5 = value (1 = High, 0 = Low)                         |   |
|   |  => 1 CPU clock cycle! Zero bus traffic! Zero address!    |   |
|   +-----------------------------------------------------------+   |
+-------------------------------------------------------------------+
```

The Silicon Breakthrough: GPIOC on Coprocessor Port p0

To eliminate this overhead entirely, Raspberry Pi engineered a custom hardware **GPIO Coprocessor (GPIOC)** directly into each ARM Cortex-M33 core on **Coprocessor Port** `p0` (Datasheet Section 3.6.1 and Section 9.9).

Instead of issuing memory load and store instructions across the system bus, the ARM core uses native ARMv8-M coprocessor instructions (`mcr` , `mcrr` , `mrc` , `mrrc`):

```
+-------------------------------------------------------------+
|            COMPLETE RP2350 GPIO SIGNAL FLOW PATH            |
|                                                             |
|  [ Cortex-M33 CPU ]                                         |
|        |                                                    |
|   (Coprocessor p0 / SIO)                                    |
|        v                                                    |
|  [ SIO Block (0xd0000000) ]                                 |
|        | (Raw Out Data & Out Enable)                        |
|        v                                                    |
|  [ IO_BANK0 Crossbar (0x40028000) ]                         |
|        | - Mux: FUNCSEL = 5 (Select SIO)                    |
|        | - Apply OUTOVER / OEOVER overrides                 |
|        v                                                    |
|  [ Voltage Level Shifter ] (1.1V DVDD Core -> 3.3V IOVDD Pad)|
|        v                                                    |
|  [ PADS_BANK0 (0x40038000) ]                                |
|        | - Check ISO latch (Must be cleared = 0)            |
|        | - Check OD bit (Must be cleared = 0)               |
|        | - Apply DRIVE strength (2/4/8/12mA) and SLEWFAST   |
|        v                                                    |
|  [ Physical Bond Pad & Package Pin ]                        |
|        |                                                    |
|        |------> External Circuit (e.g. LED, Scope, Target)  |
|        |                                                    |
|  [ Schmitt Trigger Input Buffer ]                           |
|        |                                                    |
|  [ PADS_BANK0 IE Gate ] (Must be set = 1 to pass)           |
|        v                                                    |
|  [ Voltage Level Shifter ] (3.3V IOVDD Pad -> 1.1V DVDD Core)|
|        v                                                    |
|  [ IO_BANK0 INFROMPAD ]                                     |
|        | - Apply INOVER override                            |
|        | - Route to Edge/Level Interrupt Detector           |
|        v                                                    |
|  [ SIO GPIO_IN Register / Peripheral RX ]                   |
+-------------------------------------------------------------+
```

According to **Section 3.6.1**, the GPIOC instruction set provides instantaneous, zero-latency GPIO control:

1. Single-Bit Output Enable (gpioc_bit_oe_put)

- **Opcode:** `mcrr p0, #4, Rt, Rt2, c4`
- `Rt` : General-purpose register holding the pin number (0..47).
- `Rt2` : General-purpose register holding the 1-bit value (1 = Output, 0 = Input).
- **Operation:** Atomically sets or clears the output enable for pin `Rt` in **1 single clock cycle**! No 32-bit addresses, no bit-shifts (`1 << pin`), and zero bus traffic!

2. Single-Bit Output Data (gpioc_bit_out_put)

- **Opcode:** `mcrr p0, #4, Rt, Rt2, c0`
- `Rt` : Pin number (0..47).
- `Rt2` : Value (1 = High, 0 = Low).
- **Operation:** Atomically drives pin `Rt` High or Low in 1 clock cycle.

3. Single-Pin Direct Toggling / Setting / Clearing

- Toggle pin `Rt` : `mcr p0, #5, Rt, c0, c0` (`gpioc_bit_out_xor`)
- Set pin `Rt` High: `mcr p0, #6, Rt, c0, c0` (`gpioc_bit_out_set`)

- Clear pin `Rt` Low: `mcr p0, #7, Rt, c0, c0` (`gpioc_bit_out_clr`)

4. 64-Bit Simultaneous Sampling

- **Opcode:** `mrrc p0, #0, Rt, Rt2, c0`
- Reads back all 48 GPIO pins simultaneously into register pair `Rt:Rt2` in a single clock cycle!

Part 5: Deconstructing 0x000b_integer-data-type.c

Now let us examine the production firmware in `0x000b_integer-data-type/0x000b_integer-data-type.c` . This program configures pins 16, 17, and 18 entirely through raw assembly, manipulating `PADS_BANK0` , `IO_BANK0` , and the `GPIOC` coprocessor.

Dissecting asm_init_gpio_range()

```
static void asm_init_gpio_range(void) {
    __asm volatile (
        "ldr r3, =0x40038000\n"             // address of PADS_BANK0_BASE
        "ldr r2, =0x40028004\n"             // address of IO_BANK0 GPIO0.ctrl
        "movs r0, #16\n"                    // GPIO16 (start pin)
        "init_loop:\n"                      // loop start
            "lsls r1, r0, #2\n"             // pin * 4 (pad offset)
            "adds r4, r3, r1\n"             // PADS base + offset
            "ldr  r5, [r4]\n"               // load current config
            "bic  r5, r5, #0x180\n"         // clear OD+ISO
            "orr  r5, r5, #0x40\n"          // set IE
            "str  r5, [r4]\n"               // store updated config
            "lsls r1, r0, #3\n"             // pin * 8 (ctrl offset)
            "adds r4, r2, r1\n"             // IO_BANK0 base + offset
            "ldr  r5, [r4]\n"               // load current config
            "bic  r5, r5, #0x1f\n"          // clear FUNCSEL bits [4:0]
            "orr  r5, r5, #5\n"             // set FUNCSEL = 5 (SIO)
            "str  r5, [r4]\n"               // store updated config
            "mov  r4, r0\n"                 // pin
            "movs r5, #1\n"                 // bit 1; used for OUT/OE writes
            "mcrr p0, #4, r4, r5, c4\n"     // gpioc_bit_oe_put(pin,1)
            "adds r0, r0, #1\n"             // increment pin
            "cmp  r0, #20\n"                // stop after pin 18
            "blt  init_loop\n"              // loop until r0 == 20
    );
}
```

Line-by-Line Machine Code and Silicon Execution Trace

1. `ldr r3, =0x40038000`: Loads `PADS_BANK0_BASE` into `r3`.
2. `ldr r2, =0x40028004`: Loads the address of `IO_BANK0: GPIO0_CTRL` into `r2`.
3. `movs r0, #16`: Sets loop counter starting at GPIO 16.
4. `lsls r1, r0, #2`: Multiplies pin number by 4 (16 * 4 = 64 = 0x40).
5. `adds r4, r3, r1`: Calculates the pad register address (0x40038000 + 0x40 = 0x40038040).
6. `ldr r5, [r4]`: Reads current pad configuration.
7. `bic r5, r5, #0x180`: Clears bits 7 and 8 (0x180 = 0001 1000 0000b):
   - **Bit 7 (`OD`):** Cleared to `0` -> Output driver enabled!
   - **Bit 8 (`ISO`):** Cleared to `0` -> Pad isolation latch de-asserted! The pad is connected to internal core logic!
8. `orr r5, r5, #0x40`: Sets bit 6 (0x40 = 0100 0000b):
   - **Bit 6 (`IE`):** Set to `1` -> Input buffer enabled!
9. `str r5, [r4]`: Writes new configuration back to `PADS_BANK0`. The physical pad cell is now electrically active!
10. `lsls r1, r0, #3`: Multiplies pin number by 8 (16 * 8 = 128 = 0x80).
11. `adds r4, r2, r1`: Calculates `GPIO16_CTRL` address (0x40028004 + 0x80 = 0x40028084).
12. `ldr r5, [r4]`: Reads current control register.
13. `bic r5, r5, #0x1f`: Clears bits 4:0 (`FUNCSEL`), wiping out the reset value of `31` (NULL).
14. `orr r5, r5, #5`: Sets `FUNCSEL = 5` -> Connects pin 16 to the SIO block!
15. `str r5, [r4]`: Writes back to `IO_BANK0`. The digital routing switchboard is now connected!
16. `mcrr p0, #4, r4, r5, c4`: Calls `gpioc_bit_oe_put(16, 1)`. The GPIO coprocessor on port `p0` sets the Output Enable bit in SIO in 1 single clock cycle!

Dissecting asm_blink_pin()

```
static void asm_blink_pin(uint8_t pin) {
    __asm volatile (
        "mov r4, %0\n"
        "movs r5, #0x01\n"
        "mcrr p0, #4, r4, r5, c0\n"         // Drive High
        : : "r"(pin) : "r4", "r5"
    );
    sleep_ms(500);
    __asm volatile (
        "mov r4, %0\n"
        "movs r5, #0\n"
        "mcrr p0, #4, r4, r5, c0\n"         // Drive Low
        : : "r"(pin) : "r4", "r5"
    );
    sleep_ms(500);
}
```

- When `mcrr p0, #4, r4, r5, c0` executes with `r5 = 1`, the coprocessor drives the output bit High in 1 cycle.
- The high signal flows from SIO -> `IO_BANK0` multiplexer -> Voltage Level Shifter (1.1V -> 3.3V) -> `PADS_BANK0` push-pull MOSFET driver -> Physical Pin 16 -> LED illuminates!
- When `mcrr p0, #4, r4, r5, c0` executes with `r5 = 0`, the coprocessor drives the output bit Low in 1 cycle, turning off the LED.

Part 6: Live GDB and Ghidra Dynamic & Static Reverse Engineering

The Hardware MMIO Blind Spot

When a reverse engineer opens a stripped binary or analyzes a firmware dump, disassemblers do not possess semantic knowledge of microcontroller memory maps.

In standard GDB disassembly, you encounter instructions like:

```
0x10000320 <+16>:    ldr  r3, [pc, #48]    ; =0x40038000
0x10000322 <+18>:    ldr  r2, [pc, #48]    ; =0x40028004
0x1000032c <+28>:    bic  r5, r5, #384     ; 0x180
0x10000336 <+38>:    orr  r5, r5, #5
0x1000033c <+44>:    mcrr p0, #4, r4, r5, c4
```

To an unassisted analyst, `0x40038000` is an anonymous memory address, `0x180` is an arbitrary bitmask, and `mcrr p0` looks like an invalid or proprietary instruction. This is the **Hardware MMIO Blind Spot**.

Dynamic Hardware Awareness with PyCortexMDebug in GDB

To eliminate this blind spot in live debugging, we use **PyCortexMDebug**. PyCortexMDebug ingests the official ARM CMSIS System View Description (`.svd`) file for the RP2350 (`rp2350.svd`).

Once loaded into GDB over OpenOCD and SWD:

```
(gdb) svd-load /path/to/rp2350.svd
(gdb) svd PADS_BANK0
PADS_BANK0 @ 0x40038000:
  VOLTAGE_SELECT = 0x00000000
  GPIO0          = 0x00000104  [ISO=1, DRIVE=4mA, PDE=1, SCHMITT=1]
  GPIO16         = 0x00000054  [ISO=0, OD=0, IE=1, DRIVE=4mA, PDE=1, SCHMITT=1]
```

Notice the difference! In an instant:

- We verify that `GPIO0` has `ISO=1` (pad is isolated).
- We verify that after `asm_init_gpio_range()` runs, `GPIO16` has `ISO=0` , `OD=0` , and `IE=1` !

You can also read the live hardware status of the digital crossbar switchboard:

```
(gdb) svd IO_BANK0 GPIO16_CTRL
IO_BANK0 -> GPIO16_CTRL @ 0x40028084:
  FUNCSEL = 0x05 (SIO)
  OUTOVER = 0x00 (NORMAL)
  OEOVER  = 0x00 (NORMAL)
  INOVER  = 0x00 (NORMAL)
```

GDB confirms that GPIO 16 is actively routed to the SIO block!

Static Reversing with SVD-Loader-Ghidra

In static analysis with Ghidra, decompiled code for raw MMIO looks like:

```
*(uint32_t *)(uVar3 + 0x40) = *(uint32_t *)(uVar3 + 0x40) & 0xfffffe7f | 0x40;
*(uint32_t *)(uVar2 + 0x80) = *(uint32_t *)(uVar2 + 0x80) & 0xfffffff0 | 5;
```

This decompilation is obscure. By running **SVD-Loader-Ghidra**:

1. Ghidra creates memory-mapped peripheral blocks labeled `PADS_BANK0` and `IO_BANK0`.
2. Ghidra generates C structure definitions matching the RP2350 hardware layout.
3. The decompilation transforms into clean, readable peripheral code:

```
pads_bank0->GPIO[16] = (pads_bank0->GPIO[16] & ~0x180) | 0x40;
io_bank0->GPIO[16].CTRL = (io_bank0->GPIO[16].CTRL & ~0x1f) | FUNCSEL_SIO;
```

Part 7: Summary & Hardware Register Master Cheat Sheet

Architectural Summary

1. **Hardware Split:** RP2350 enforces a strict division between electrical physics (`PADS_BANK0`) and digital logic (`IO_BANK0`).
2. **PADS_BANK0 (`0x40038000`):** Manages analog pad cells. Controls `ISO` (isolation latch), `OD` (output disable), `IE` (input enable), `DRIVE` (2/4/8/12 mA), `PUE` / `PDE` (pulls and Bus Keeper), `SCHMITT`, and `SLEWFAST`.
3. **IO_BANK0 (`0x40028000`):** Manages digital crossbar routing. Controls `FUNCSEL` (functions 0..31; 5 = SIO), logic overrides (`OUTOVER` , `OEOVER` , `INOVER` , `IRQOVER`), and edge/level interrupt generation.
4. **SIO (`0xd0000000`):** Single-Cycle I/O memory-mapped registers for 32-bit atomic bit set, clear, toggle, and read.
5. **GPIOC (Coprocessor Port `p0`):** Native ARMv8-M coprocessor hardware enabling 1-cycle single-bit output and output-enable manipulation (`mcrr p0, #4`), single-pin toggling (`mcr p0, #5`), and 64-bit sampling (`mrrc p0, #0`) with zero memory bus overhead.

Hardware Register Reference Table

Subsystem	Base Address	Register / Offset	Key Bitfields & Silicon Function
PADS_BANK0	`0x40038000`	`VOLTAGE_SELECT` (`+0x00`)	Bit 0: Bank voltage select (`0 = 3.3V` , `1 = 1.8V`)
PADS_BANK0	`0x40038000`	`GPIOx` (`+0x04 + 4*x`)	Bit 8: `ISO` (Isolation Latch, reset=1) Bit 7: `OD` (Output Disable) Bit 6: `IE` (Input Enable, reset=0) Bits 5:4: `DRIVE` (`00=2mA` , `01=4mA` , `10=8mA` , `11=12mA`) Bit 3: `PUE` (Pull-Up Enable) Bit 2: `PDE` (Pull-Down Enable) Bit 1: `SCHMITT` (Hysteresis Enable) Bit 0: `SLEWFAST` (Slew Rate Control)
IO_BANK0	`0x40028000`	`GPIOx_STATUS` (`+0x000 + 8*x`)	Bit 17: `INFROMPAD` (Raw input level) Bit 13: `OETOPAD` (Effective Output Enable) Bit 9: `OUTTOPAD` (Effective Output Data) Bit 26: `IRQTOPROC` (Interrupt status)

Subsystem	Base Address	Register / Offset	Key Bitfields & Silicon Function
IO_BANK0	`0x40028000`	`GPIOx_CTRL` (`+0x004 + 8*x`)	Bits 4:0: `FUNCSEL` (`0=JTAG`, `1=SPI`, `2=UART`, `3=I2C`, `4=PWM`, `5=SIO`, `6=PIO0`, `31=NULL`) Bits 13:12: `OUTOVER` Bits 15:14: `OEOVER` Bits 17:16: `INOVER` Bits 29:28: `IRQOVER`
SIO	`0xd0000000`	`GPIO_OUT` (`+0x10`)	Bits 31:0: Output data values
SIO	`0xd0000000`	`GPIO_OUT_SET` (`+0x14`)	Bits 31:0: Atomic bit set High
SIO	`0xd0000000`	`GPIO_OUT_CLR` (`+0x18`)	Bits 31:0: Atomic bit clear Low
SIO	`0xd0000000`	`GPIO_OUT_XOR` (`+0x1c`)	Bits 31:0: Atomic bit toggle
SIO	`0xd0000000`	`GPIO_OE` (`+0x20`)	Bits 31:0: Output enable (`1 = Out, 0 = In`)
SIO	`0xd0000000`	`GPIO_IN` (`+0x04`)	Bits 31:0: Direct input sample (Requires `IE=1`)
GPIOC	Port `p0`	`mcrr p0, #4, Rt, Rt2, c4`	Single-pin atomic output enable write (`gpioc_bit_oe_put`)
GPIOC	Port `p0`	`mcrr p0, #4, Rt, Rt2, c0`	Single-pin atomic output data write (`gpioc_bit_out_put`)
GPIOC	Port `p0`	`mcr p0, #5, Rt, c0, c0`	Single-pin atomic output toggle (`gpioc_bit_out_xor`)
GPIOC	Port `p0`	`mcr p0, #6, Rt, c0, c0`	Single-pin atomic output set (`gpioc_bit_out_set`)
GPIOC	Port `p0`	`mcr p0, #7, Rt, c0, c0`	Single-pin atomic output clear (`gpioc_bit_out_clr`)
GPIOC	Port `p0`	`mrrc p0, #0, Rt, Rt2, c0`	64-bit dual-word simultaneous GPIO sample